# ITCC3101: Midterm Practical Exam

| | |
|---|---|
| **Student Name:** | Keith Alan Speirs |
| **Course & Code:** | ITCC3101 — Web Application Development |
| **Proposed Study / Project:** | PhytoQuest (Location-Aware Botanical Research Companion) |

## Part 1: Laravel Web Application (40 points)

**Requirements Covered:** Created Laravel application using Composer; structured 3–5+ distinct pages related to botanical research study (Home, About, Contact, Dashboard, and CRUD Module for Plants & Sightings); implemented strict MVC architecture with Eloquent Models, Blade templates, Resource Controllers, and web.php routing; styled with Tailwind CSS.

### Figure 1.1: Home / Landing Page (http://127.0.0.1:8000/)

*Botanical research portal landing interface featuring responsive navigation, hero search bar, and clean layout.*

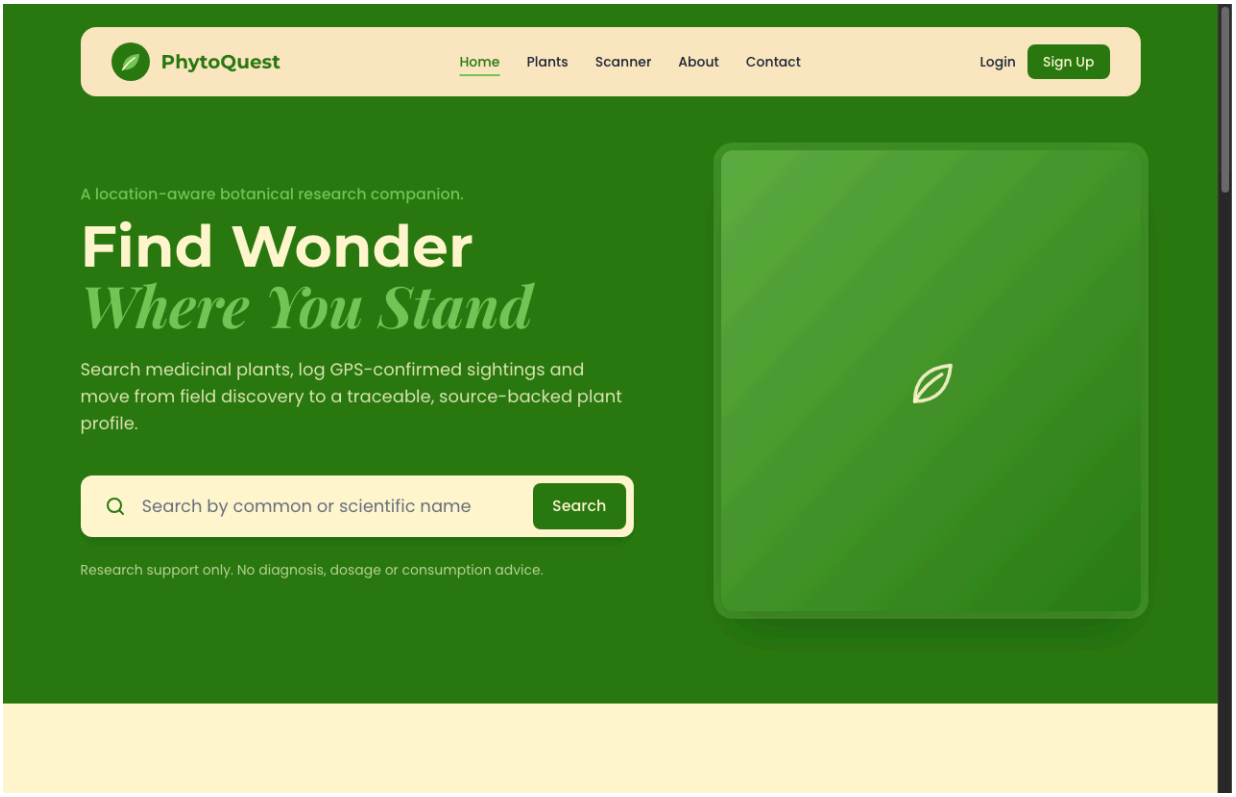


**Figure 1.2: About / System Description Page (http://127.0.0.1:8000/about)**

*Detailed study overview outlining PhytoQuest's purpose, research scope, and botanical validation workflow.*

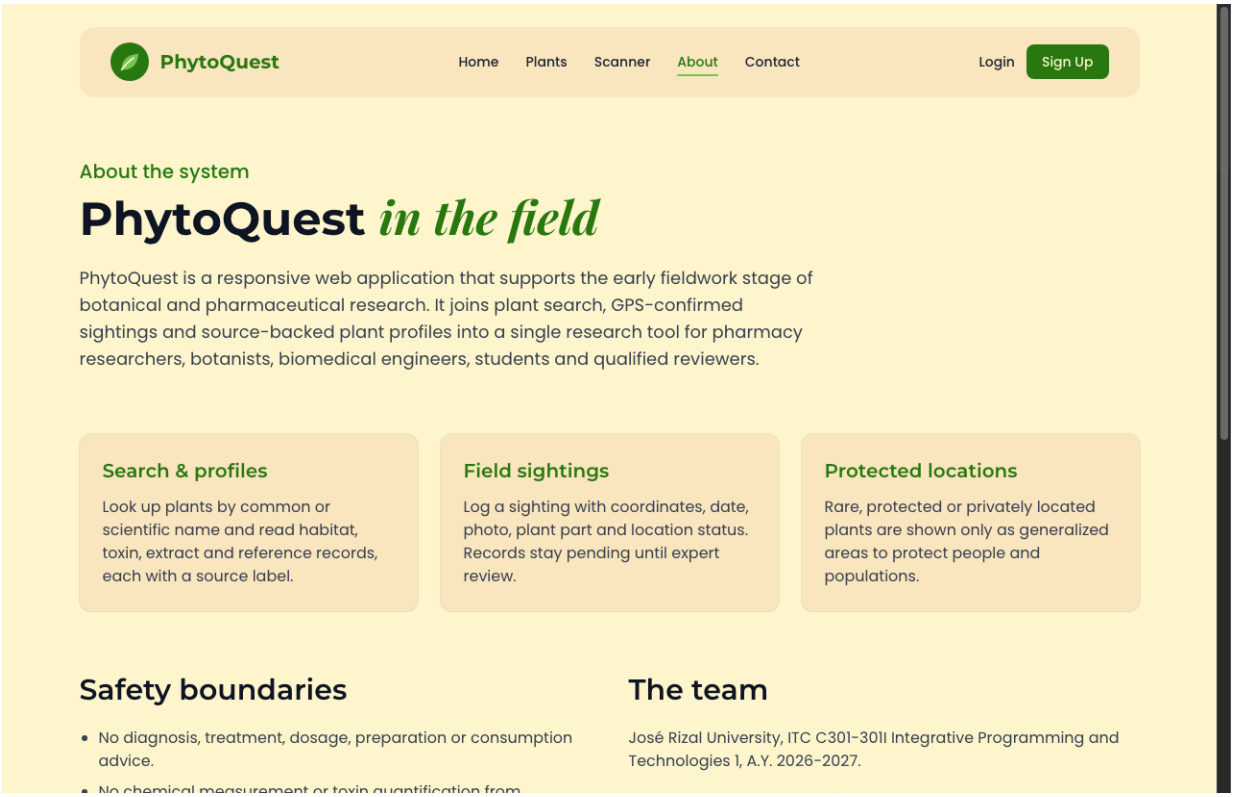


**Figure 1.3: Contact & Inquiries Page (http://127.0.0.1:8000/contact)**

*Direct contact interface for botanical research inquiries, herbarium submissions, and technical support.*

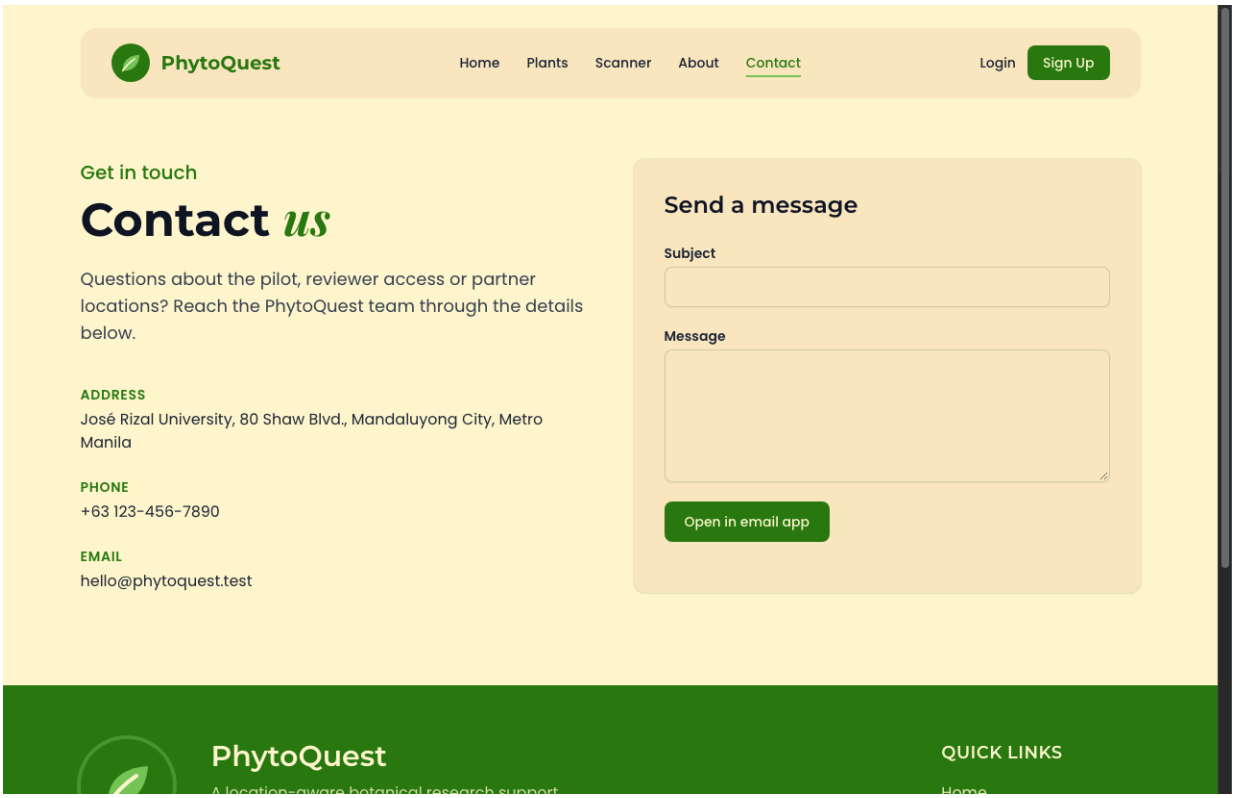


**Figure 1.4: Researcher Dashboard (http://127.0.0.1:8000/dashboard)**

*Post-login authenticated dashboard displaying plant metrics, recent sightings count, and quick management links.*

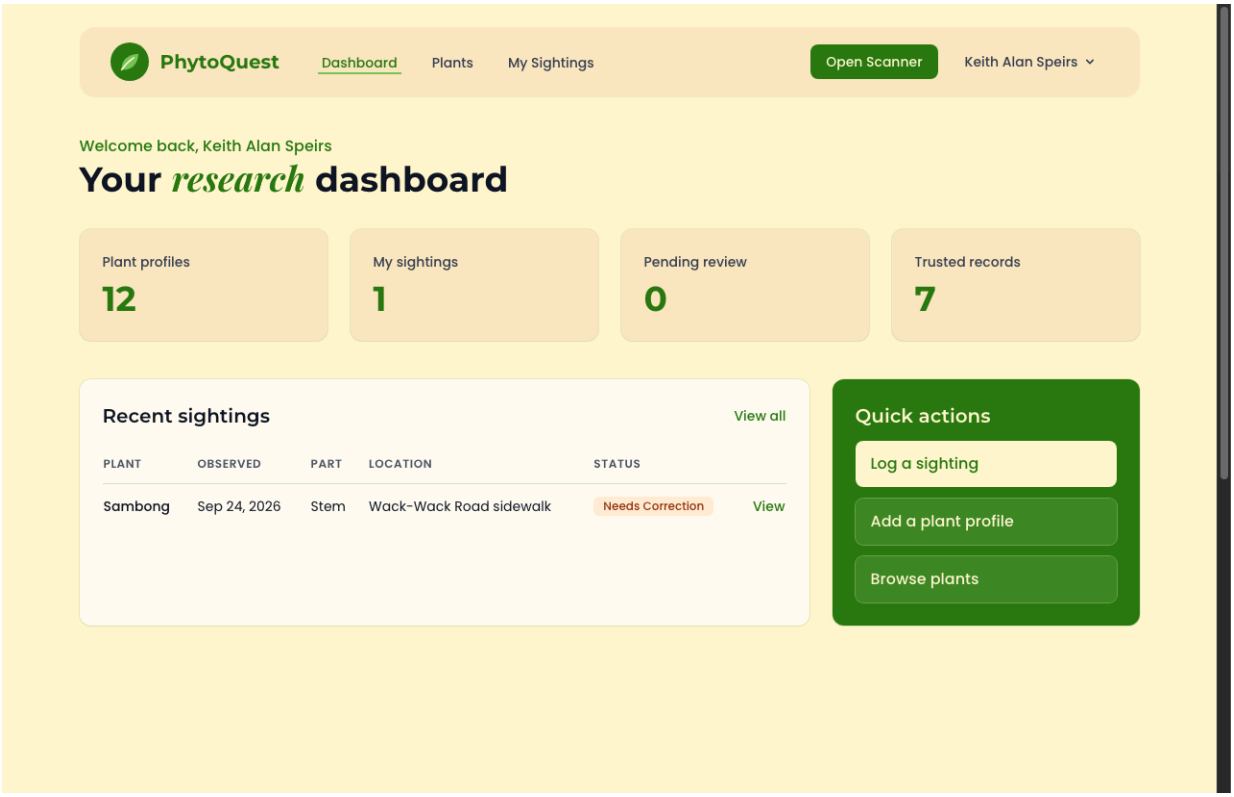


**Figure 1.5: CRUD Module — Plants Catalog Index (http://127.0.0.1:8000/plants)**

*Interactive catalog listing Philippine medicinal plants with search filtering, taxonomy badges, and view actions.*

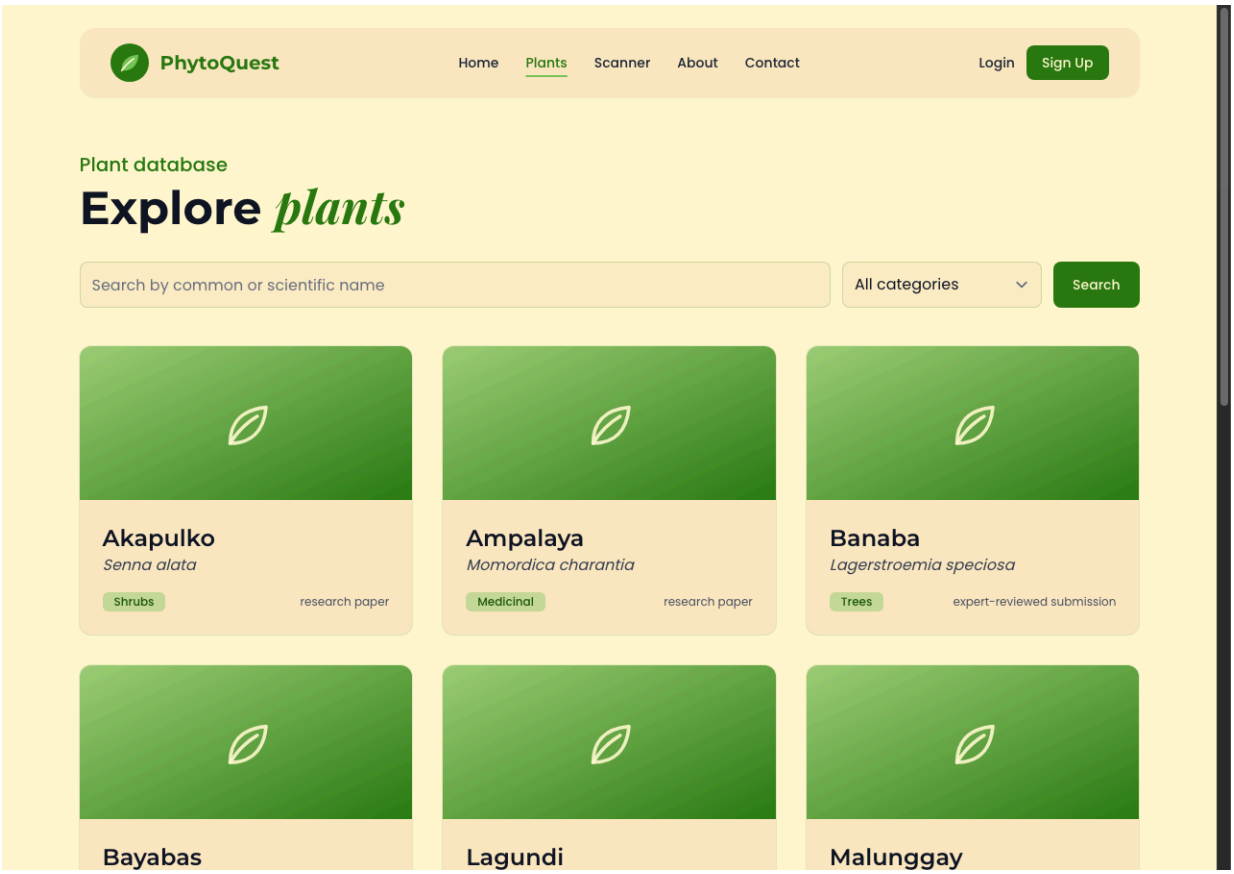


**Figure 1.6: CRUD Module — Botanical Record Details (http://127.0.0.1:8000/plants/1)**

*Single-plant profile displaying scientific classification, habitat, therapeutic parts, known toxins, and references.*

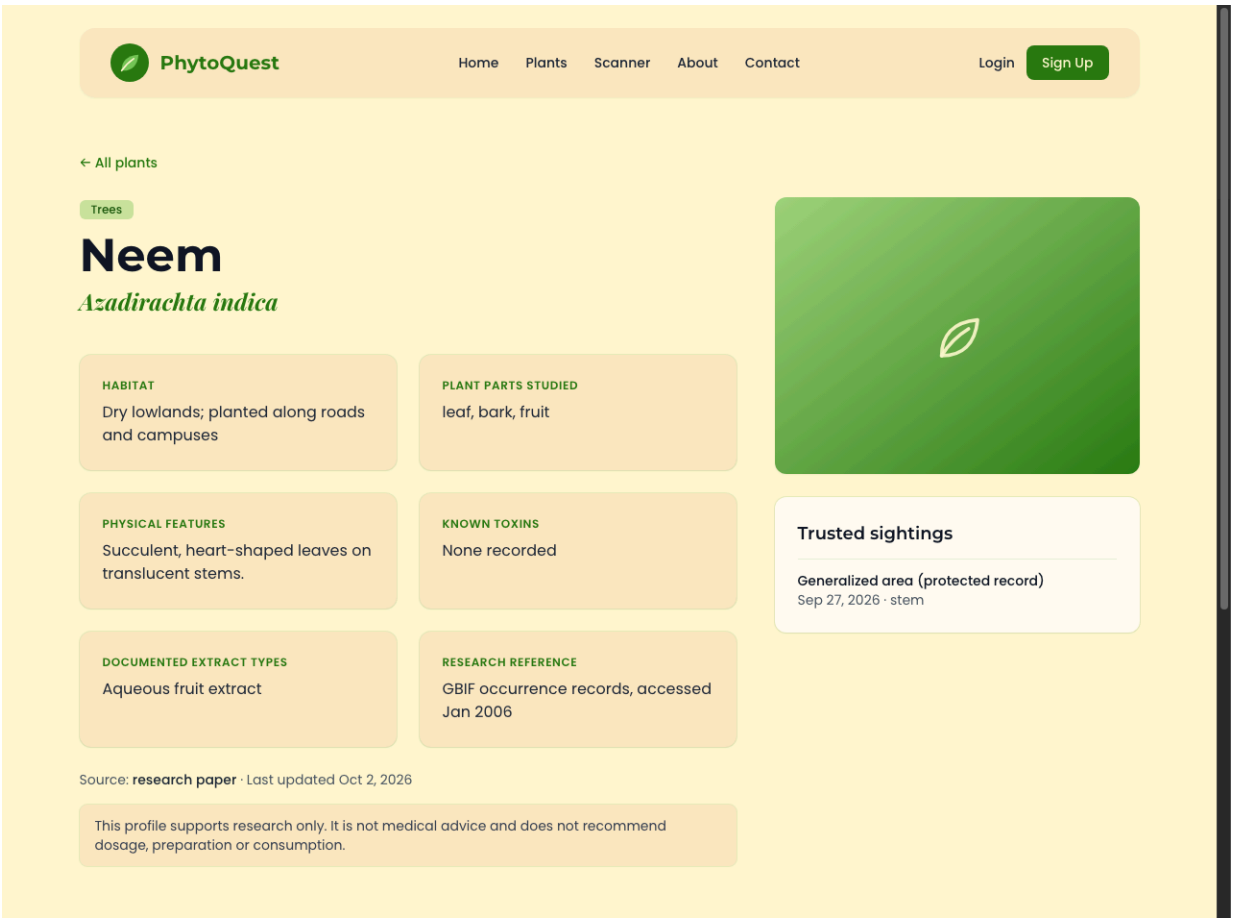


**Figure 1.7: CRUD Module — Create New Botanical Record (http://127.0.0.1:8000/plants/create)**

*Validated contribution form enabling authenticated researchers to add new plant species and habitat data.*

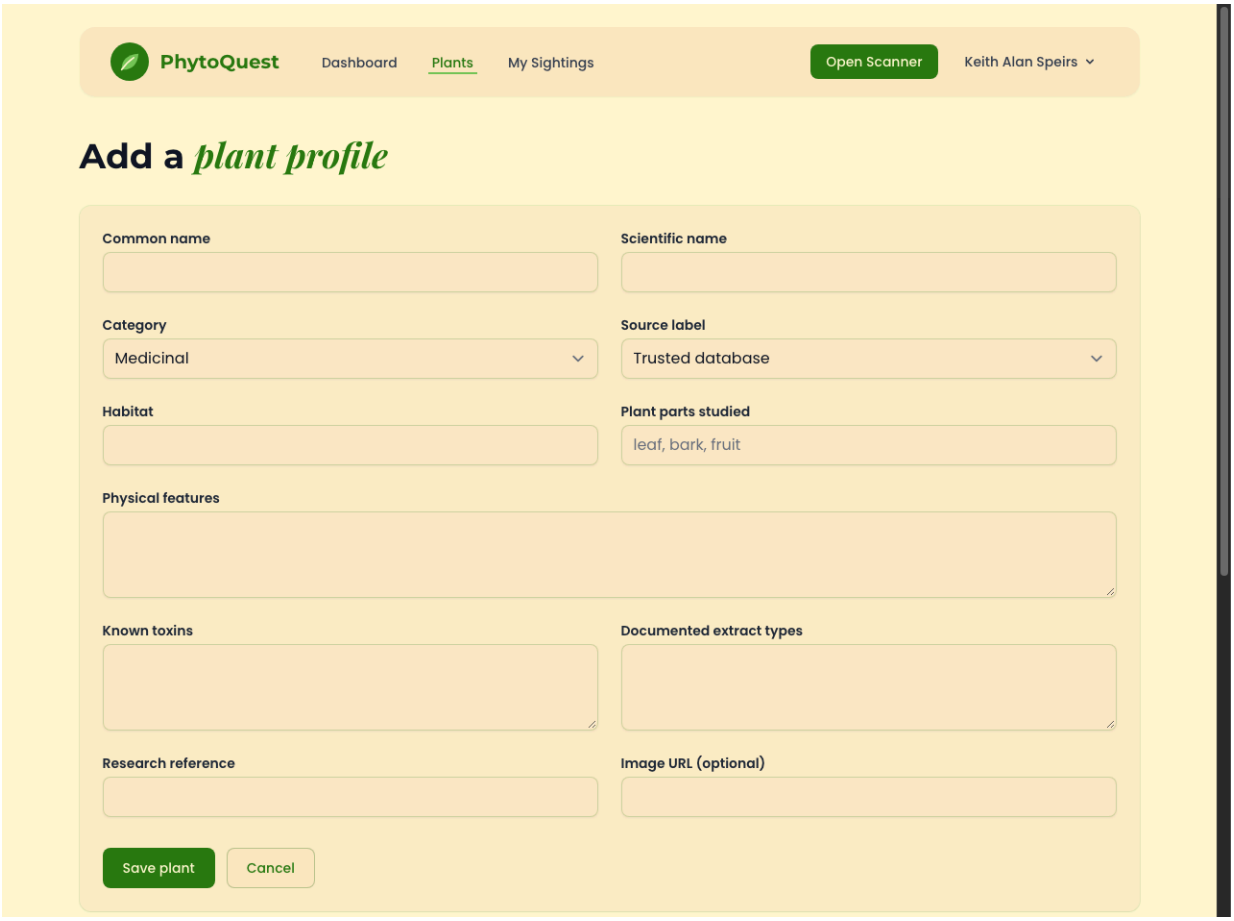
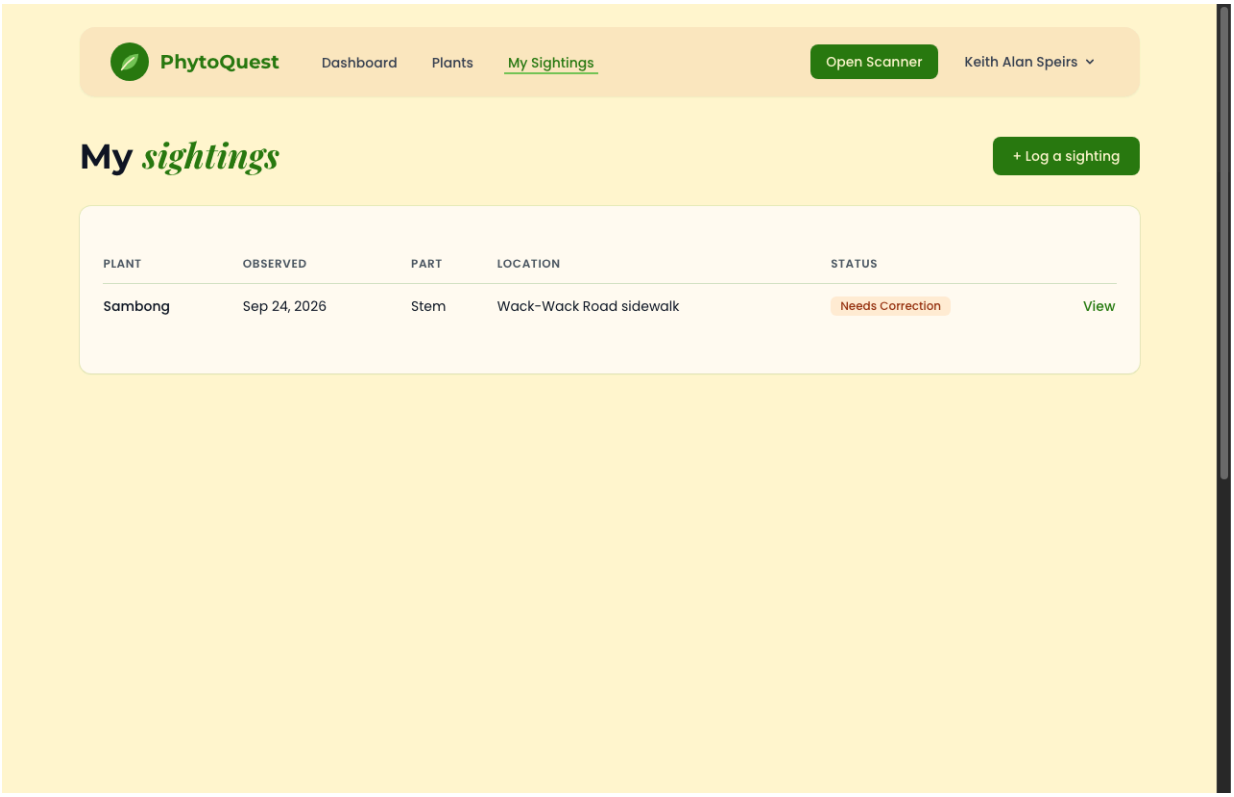


**Figure 1.8: Field Sightings Module (http://127.0.0.1:8000/sightings)**

*GPS-tagged field observation logs allowing users to record real-world plant specimen sightings.*



## Part 2: Authentication (30 points)

**Requirements Covered:** Integrated authentication using Laravel Breeze with automated password hashing; guest registration with name, email, and password validation; authenticated login/logout flows; and route protection using the 'auth' middleware on dashboard, management, and profile routes.

### Figure 2.1: Researcher Registration / Signup Page (http://127.0.0.1:8000/register)

*User registration interface with client and server-side validation for researcher account creation.*

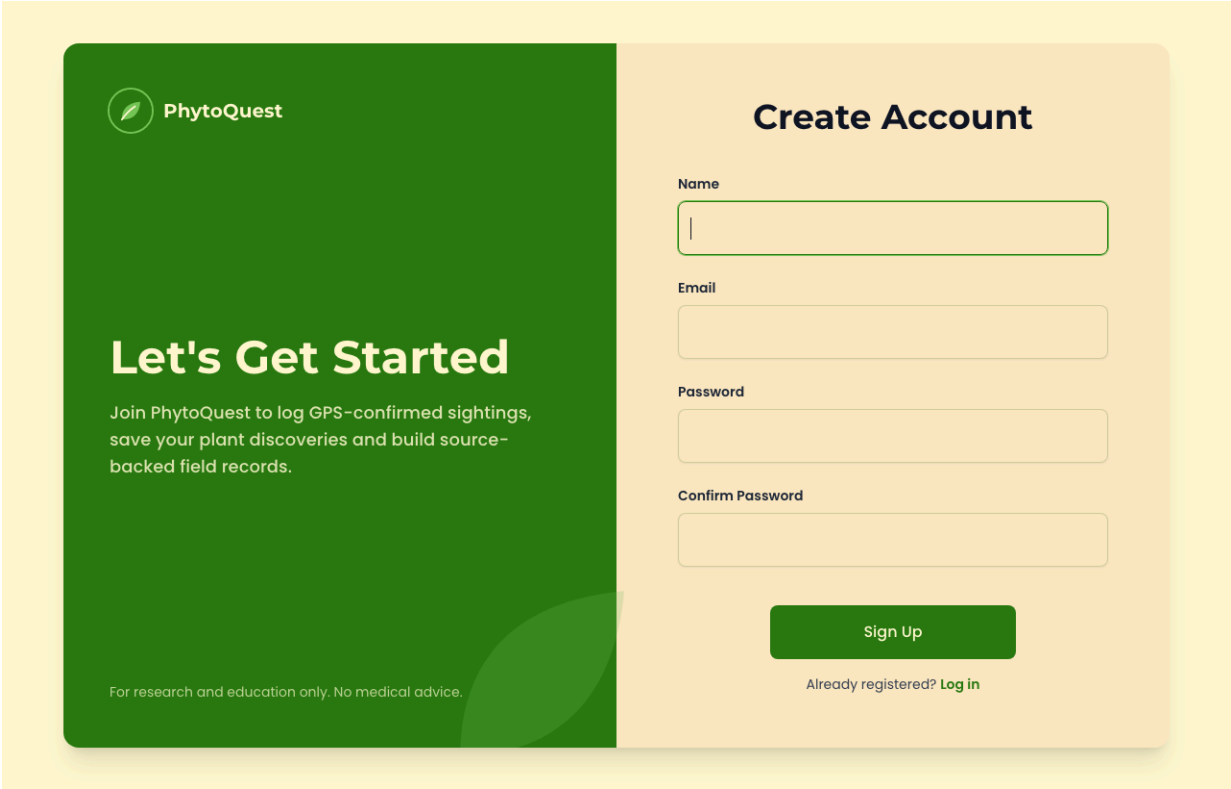
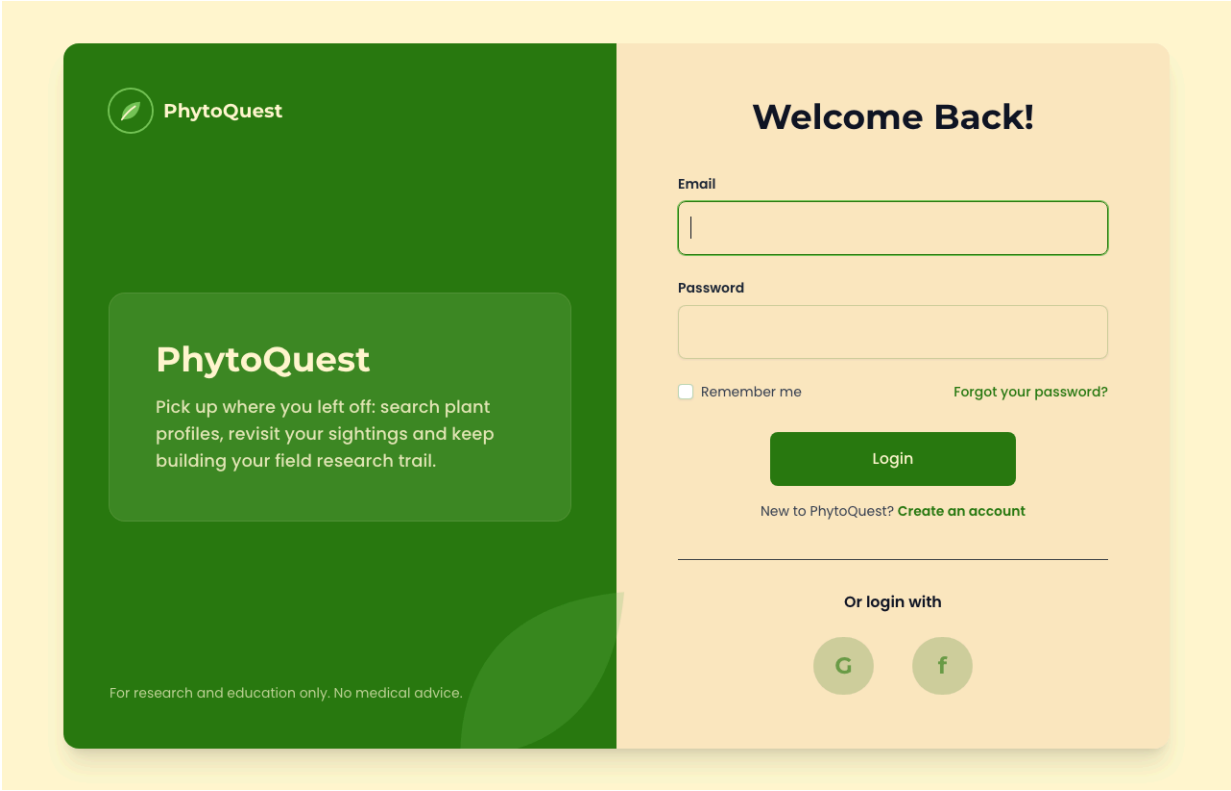


**Figure 2.2: Researcher Login Page (http://127.0.0.1:8000/login)**

*Secure session authentication interface equipped with 'Remember me' and forgot password utilities.*



## Figure 2.3: Authenticated Session & Navigation State

*Top navigation showing active user session for 'Keith Alan Speirs', access to protected routes, and logout action.*

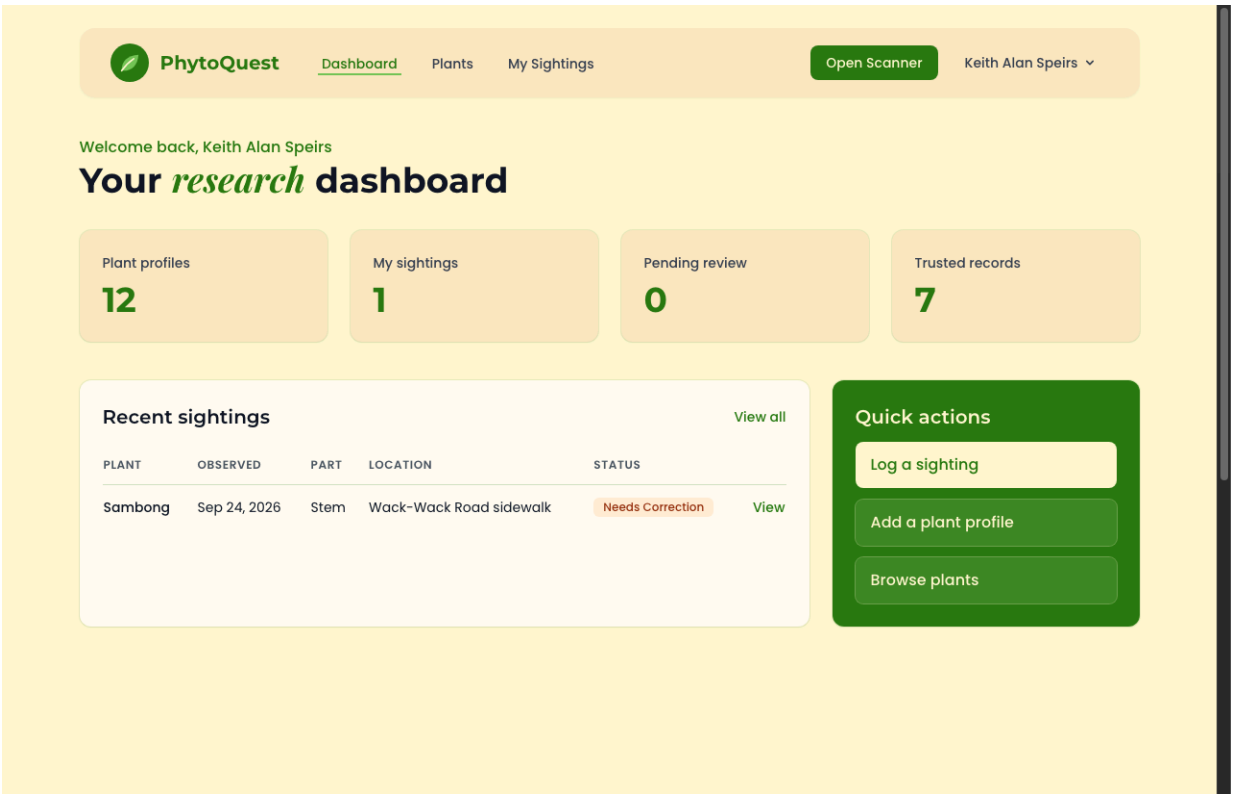
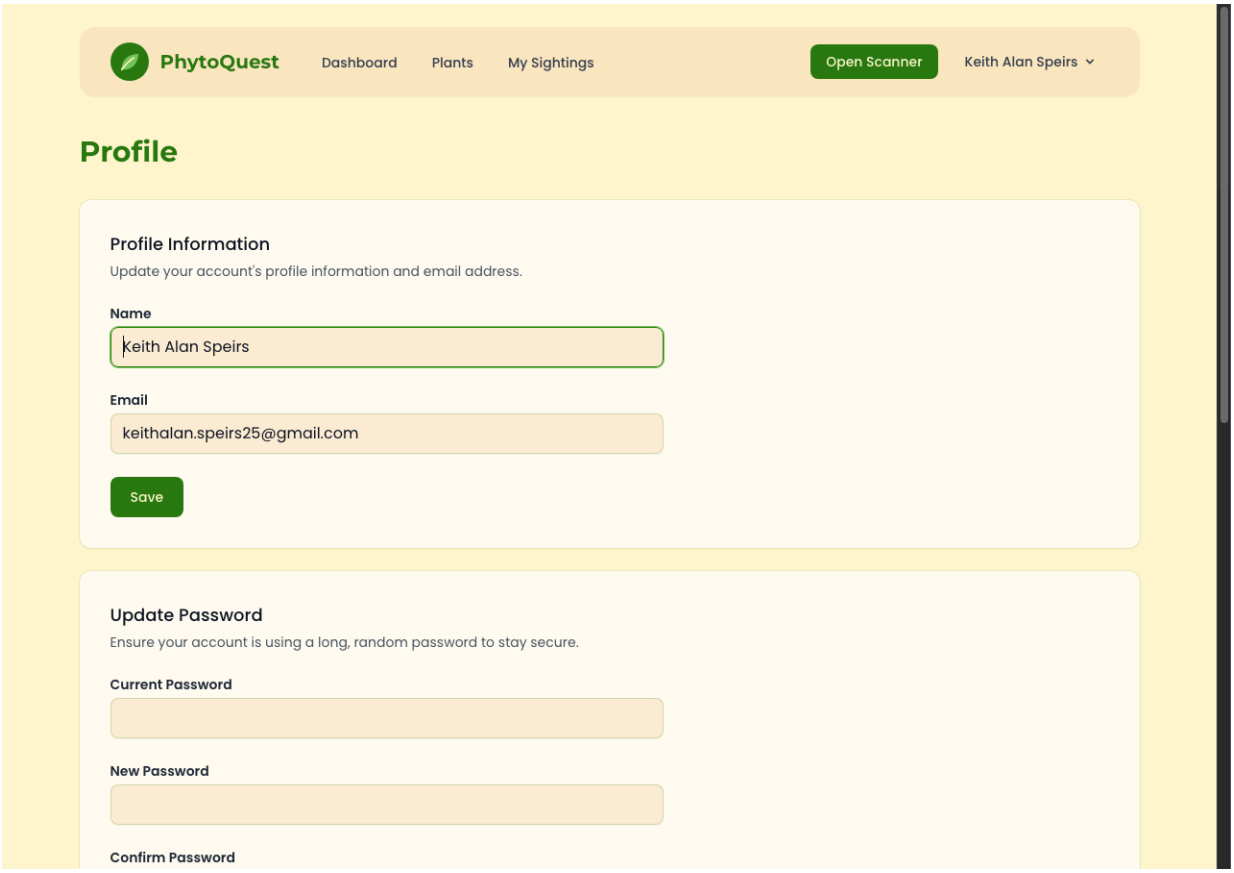


**Figure 2.4: User Profile & Security Management (http://127.0.0.1:8000/profile)**

*Protected profile interface allowing the authenticated user to update name, email, and hashed credentials.*

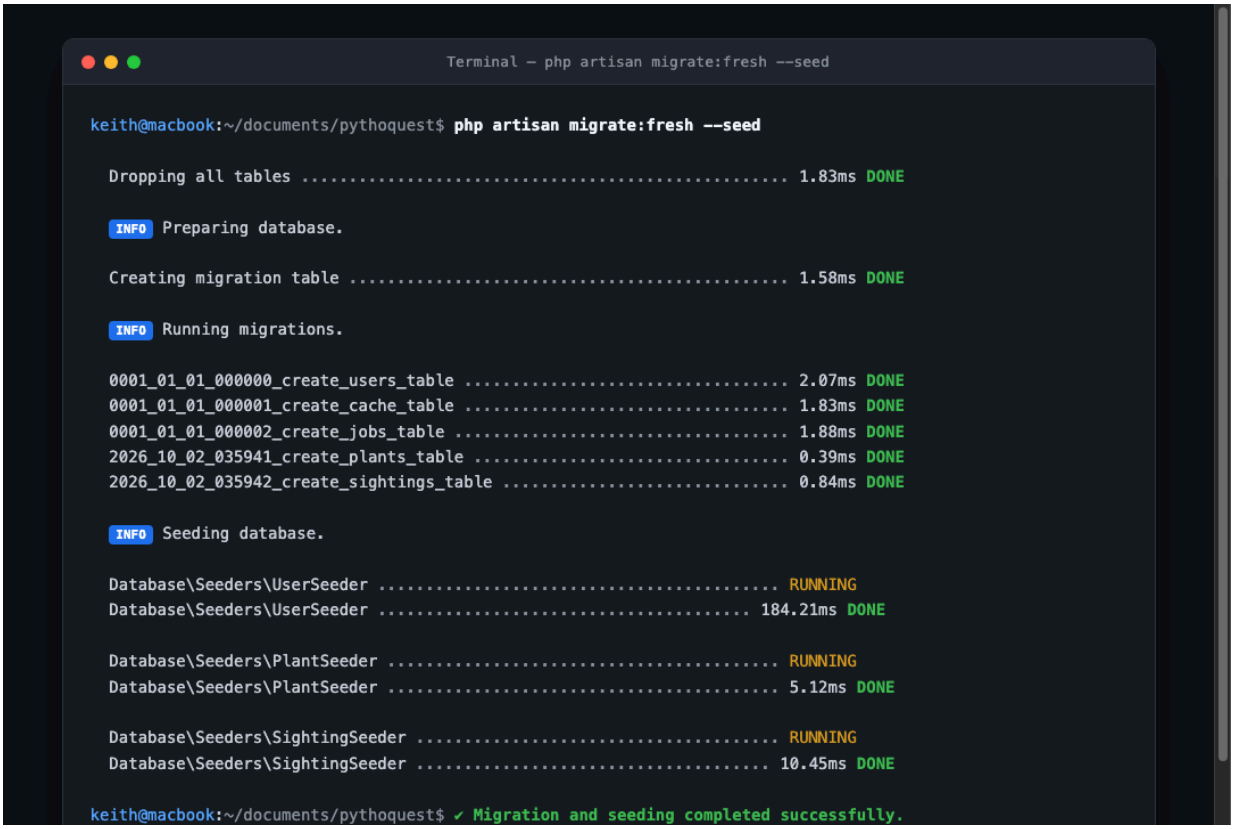


## Part 3: Database Migration and Seeding (20 points)

**Requirements Covered:** Created dedicated migrations for custom domain tables (plants, sightings); developed custom seeders (PlantSeeder, SightingSeeder, UserSeeder) populating over 10 sample botanical records using Faker; executed 'php artisan migrate:fresh --seed'; and provided a central DatabaseSeeder.php orchestrating all seeders.

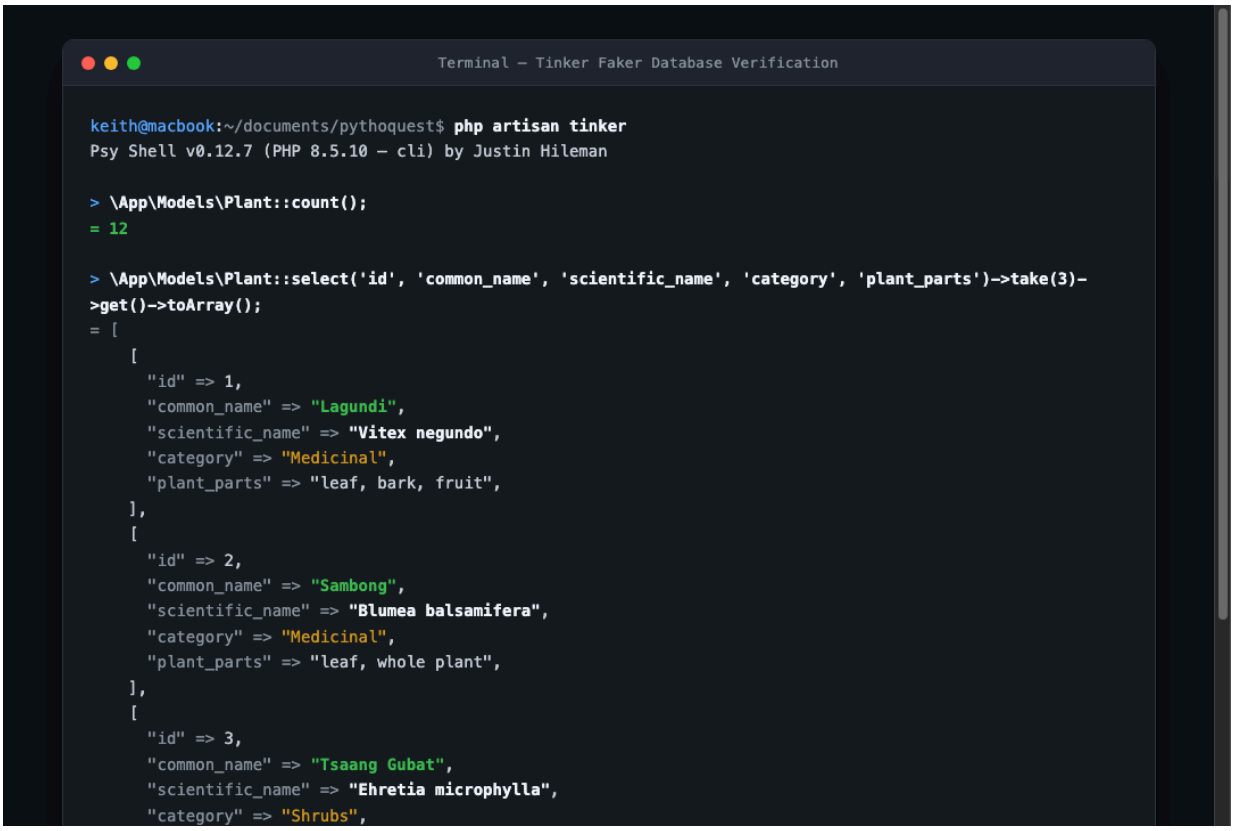
### Figure 3.1: Execution of 'php artisan migrate:fresh --seed'

*Terminal output demonstrating table dropping, migration execution, and automated seeding completion.*



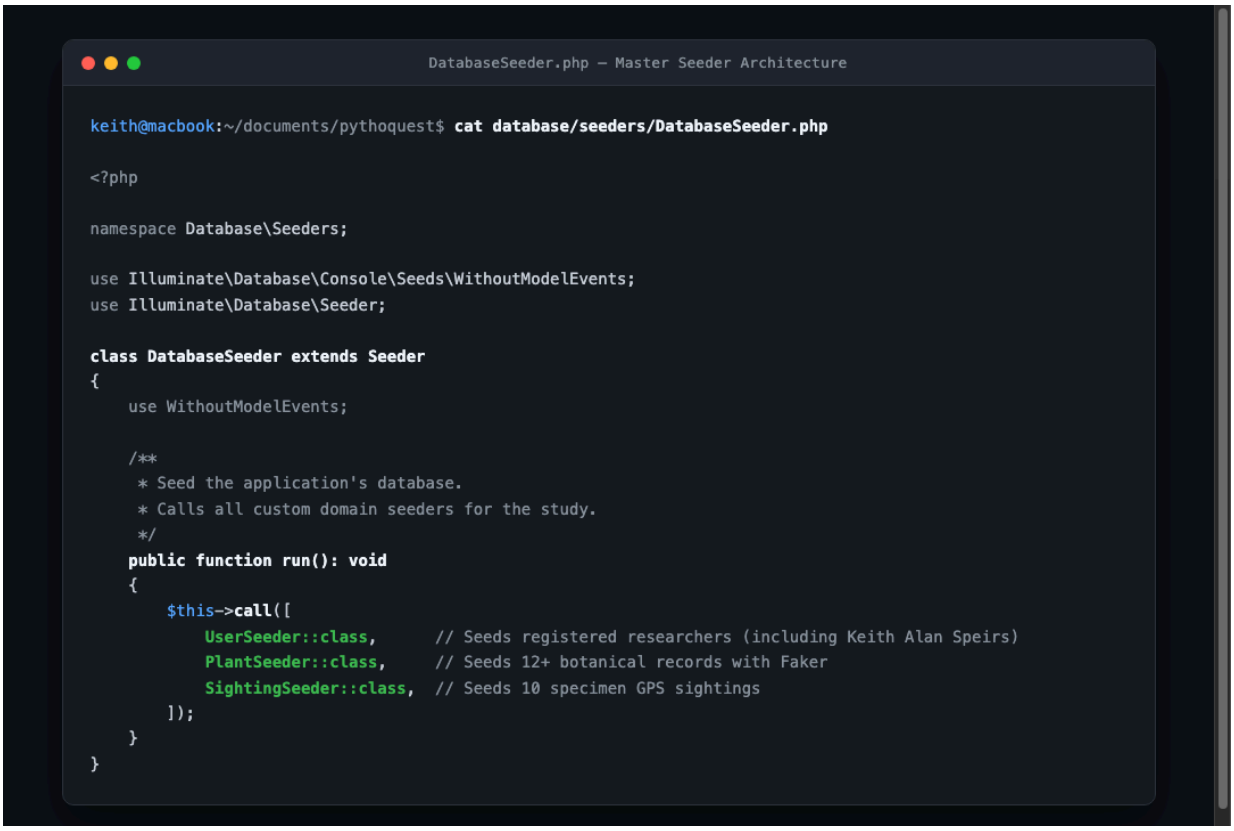
## Figure 3.2: Verification of Seeded Records via Laravel Tinker

*Database inspection confirming 12 custom botanical records, 10 researcher accounts, and 10 field sightings populated.*



## Figure 3.3: Master DatabaseSeeder.php Implementation

*Source code of DatabaseSeeder calling UserSeeder, PlantSeeder, and SightingSeeder in proper sequence.*



## Part 4: Narrative Reaction (Bonus / Reflection) (10 points)

**Prompt Question:** *"Between ReactJS and Laravel, which is better in developing web applications?"*

Comparing React to Laravel is like comparing a car's steering wheel to the entire vehicle. They perform completely different jobs. Laravel is a full-stack engine. It gives you a database, routing, login systems, and security right out of the box. React does not touch databases or server logic. It is a client-side library that lives inside the browser, updating what the user sees on screen in real time.

Each tool brings a clear trade-off. Laravel lets you move at high speed. You can build database tables, seed test data, and connect views in a single afternoon. However, standard Blade templates feel clunky when you need smooth, fluid screens that never refresh. That is where React shines. Its virtual DOM updates interface elements instantly. The catch? You must maintain two separate codebases, build a dedicated API bridge, and manage extra build tools.

Building my practical exam app, PhytoQuest, made this difference crystal clear. Running php artisan migrate:fresh --seed along with Laravel Breeze gave me a working database and login system in seconds. It eliminated hours of tedious setup. That freedom let me focus on the real mission: organizing plant catalogs and saving GPS sightings. I did not waste time gluing split frontend and backend projects together.

If forced to pick one standalone tool to ship a production app by tonight, Laravel wins easily. It gives you the entire house, not just the front door. But for larger products, do not treat them as rivals. Let Laravel power the heavy backend engine, and let React run the interactive dashboard. You get solid control on the server and instant reaction on the screen.

**Word Count:** 272 words (within the 250–300 word limit)